Files 파일 처리

개요

Boxo의 Files 모듈은 IPFS에서 다양한 타입의 파일과 디렉토리를 처리하기 위한 통합된 인터페이스와 유틸리티를 제공합니다. 이 모듈은 로컬 파일 시스템, 웹 URL, 메모리 내 데이터 등 다양한 소스로부터 파일을 읽고 처리할 수 있는 추상화 계층을 제공합니다.

주요 개념

파일 인터페이스 계층구조

Files 모듈은 다음과 같은 인터페이스 계층구조를 가집니다:

```
Node (기본 인터페이스)
├── File (일반 파일)
├── Directory (디렉토리)
└── 특수 파일 타입들
```

핵심 인터페이스

- **Node**: 모든 파일 객체의 기본 인터페이스
- **File**: 읽기 가능한 일반 파일
- **Directory**: 디렉토리와 그 내용을 나타내는 인터페이스
- **DirEntry**: 디렉토리 항목 정보

- **DirIterator**: 디렉토리 내용을 순회하는 반복자

기본 사용법

메모리에서 파일 생성

```
package main

import (
"fmt"
"io"

"github.com/ipfs/boxo/files"
)

func  createMemoryFile() {
// 바이트 슬라이스로부터 파일 생성
content := []byte("안녕하세요, IPFS Files 모듈입니다!")
file := files.NewBytesFile(content)
defer  file.Close()

// 파일 정보 확인
size, err := file.Size()
if  err != nil {
```

```
fmt.Printf("크기 가져오기 오류: %v\n", err)
return
}
fmt.Printf("파일 크기: %d 바이트\n", size)

// 파일 내용 읽기
data, err := io.ReadAll(file)
if  err != nil {
fmt.Printf("파일 읽기 오류: %v\n", err)
return
}
fmt.Printf("파일 내용: %s\n", string(data))
}
```

로컬 파일 시스템에서 파일 읽기

```
package main

import (
"fmt"
"io"
"os"
```

```
"github.com/ipfs/boxo/files"
)

func readLocalFile(filePath string) error {
// 파일 정보 가져오기
stat, err := os.Stat(filePath)
if err != nil {
return fmt.Errorf("파일 정보 가져오기 실패: %v", err)
}

// SerialFile로 파일 생성 (숨김 파일 포함)
node, err := files.NewSerialFile(filePath, true, stat)
if err != nil {
return fmt.Errorf("파일 생성 실패: %v", err)
}
defer node.Close()

// 파일 타입 확인
if file := files.ToFile(node); file != nil {
// 일반 파일 처리
fmt.Printf("일반 파일: %s\n", filePath)
fmt.Printf("크기: %d 바이트\n", stat.Size())
fmt.Printf("수정 시간: %v\n", file.ModTime())
```

```
fmt.Printf("권한: %v\n", file.Mode())


// 파일 내용 읽기 (처음 100바이트만)
buffer := make([]byte, 100)
n, err := file.Read(buffer)
if err != nil && err != io.EOF {
return fmt.Errorf("파일 읽기 실패: %v", err)
}
fmt.Printf("내용 미리보기: %s\n", string(buffer[:n]))


} else if dir := files.ToDir(node); dir != nil {
// 디렉토리 처리
fmt.Printf("디렉토리: %s\n", filePath)
return listDirectory(dir)
}


return nil
}
```

디렉토리 내용 순회

```
func listDirectory(dir files.Directory) error {
// 디렉토리 항목 반복자 가져오기
```

```
entries := dir.Entries()
fmt.Println("디렉토리 내용:")
for entries.Next() {
entry := entries
// 항목 정보 출력
fmt.Printf("- %s", entry.Name())
node := entry.Node()
if size, err := node.Size(); err == nil {
fmt.Printf(" (크기: %d)", size)
}
// 파일 타입 확인
if files.ToFile(node) != nil {
fmt.Printf(" [파일]")
} else if files.ToDir(node) != nil {
fmt.Printf(" [디렉토리]")
}
fmt.Printf(" 권한: %v", node.Mode())
fmt.Printf(" 수정시간: %v\n", node.ModTime())
}
// 반복 중 오류 확인
if err := entries.Err(); err != nil {
return fmt.Errorf("디렉토리 순회 오류: %v", err)
}
```

```
return nil
}
```

다양한 파일 타입 처리

웹 파일 처리

```
package main

import (
"fmt"
"io"
"net/url"

"github.com/ipfs/boxo/files"
)

func readWebFile(urlStr string) error {
// URL 파싱
u, err := url.Parse(urlStr)
if err != nil {
return fmt.Errorf("URL 파싱 실패: %v", err)
}
```

```go
// 웹 파일 생성
webFile := files.NewWebFile(u)
defer webFile.Close()

// 파일 정보 확인
fmt.Printf("웹 파일: %s\n", urlStr)
fmt.Printf("수정 시간: %v\n", webFile.ModTime())
fmt.Printf("권한: %v\n", webFile.Mode())

// 파일 크기 확인
size, err := webFile.Size()
if err != nil {
fmt.Printf("크기 확인 실패: %v\n", err)
} else {
fmt.Printf("크기: %d 바이트\n", size)
}

// 내용 읽기 (처음 500바이트만)
buffer := make([]byte, 500)
n, err := webFile.Read(buffer)
if err != nil && err != io.EOF {
return fmt.Errorf("웹 파일 읽기 실패: %v", err)
```

```
}

fmt.Printf("내용 미리보기:\n%s\n", string(buffer[:n]))
return  nil
}
```

리더 파일 생성

```
package main

import (
"fmt"
"io"
"os"
"strings"

"github.com/ipfs/boxo/files"
)

func  createReaderFile() error {
// 문자열 리더로부터 파일 생성
content := "이것은 리더로부터 생성된 파일입니다."
reader := strings.NewReader(content)
```

```
// 파일 정보 생성 (선택사항)
stat := &mockFileInfo{
name: "example.txt",
size: int64(len(content)),
mode: 0644,
modTime: time.Now(),
}
// ReaderFile 생성
file := files.NewReaderStatFile(reader, stat)
defer file.Close()

// 파일 정보 확인
fmt.Printf("파일명: %s\n", stat.Name())
fmt.Printf("크기: %d 바이트\n", stat.Size())
fmt.Printf("권한: %v\n", file.Mode())

// 내용 읽기
data, err := io.ReadAll(file)
if err != nil {
return fmt.Errorf("파일 읽기 실패: %v", err)
}

fmt.Printf("내용: %s\n", string(data))
```

```
return nil
}

// 모의 FileInfo 구현
type mockFileInfo struct {
name string
size int64
mode os.FileMode
modTime time.Time
}

func (m *mockFileInfo) Name() string { return m.name }
func (m *mockFileInfo) Size() int64 { return m.size }
func (m *mockFileInfo) Mode() os.FileMode { return m.mode }
func (m *mockFileInfo) ModTime() time.Time { return m.modTime }
func (m *mockFileInfo) IsDir() bool { return m.mode.IsDir() }
func (m *mockFileInfo) Sys() interface{} { return nil }
```

파일 메타데이터 관리

파일 권한 처리

```
package main
```

```go
import (
"fmt"
"os"

"github.com/ipfs/boxo/files"
)

func handleFilePermissions(node files.Node) {
// 현재 권한 확인
mode := node.Mode()
fmt.Printf("현재 권한: %v (8진수: %o)\n", mode, mode)

// Unix 스타일 권한으로 변환
unixPerms := files.ModePermsToUnixPerms(mode)
fmt.Printf("Unix 권한: %o\n", unixPerms)

// 기본 권한 가져오기
defaultPerms := files.UnixPermsOrDefault(node)
fmt.Printf("기본 권한: %o\n", defaultPerms)

// 파일 타입별 권한 확인
switch node.(type) {
```

```
case files.File:
fmt.Println("파일 타입: 일반 파일")
case files.Directory:
fmt.Println("파일 타입: 디렉토리")
default:
fmt.Println("파일 타입: 기타")
}
}
```

메타데이터 업데이트

```
package main

import (
"fmt"
"os"
"time"

"github.com/ipfs/boxo/files"
)

func updateFileMetadata(filePath string) error {
// 새로운 수정 시간 설정
```

```
newModTime := time.Now().Add(-24 * time.Hour) // 하루 전
// 수정 시간 업데이트
err := files.UpdateModTime(filePath, newModTime)
if err != nil {
return fmt.Errorf("수정 시간 업데이트 실패: %v", err)
}
fmt.Printf("수정 시간이 %v로 업데이트되었습니다\n", newModTime)

// 파일 권한 업데이트
newMode := os.FileMode(0755)
err = files.UpdateFileMode(filePath, newMode)
if err != nil {
return fmt.Errorf("파일 권한 업데이트 실패: %v", err)
}
fmt.Printf("파일 권한이 %v로 업데이트되었습니다\n", newMode)

// Unix 스타일 권한으로 업데이트
unixMode := uint32(0644)
err = files.UpdateFileModeUnix(filePath, unixMode)
if err != nil {
return fmt.Errorf("Unix 권한 업데이트 실패: %v", err)
}
fmt.Printf("Unix 권한이 %o로 업데이트되었습니다\n", unixMode)
```

```
return nil
}
```

파일 필터링

파일 필터 생성 및 사용

```
package main

import (
"fmt"
"os"

"github.com/ipfs/boxo/files"
)

func useFileFilter(dirPath string) error {
// .gitignore 스타일 규칙으로 필터 생성
rules := []string{
"*.tmp", // 임시 파일 제외
"*.log", // 로그 파일 제외
".DS_Store", // macOS 시스템 파일 제외
```

```
"node_modules/", // Node.js 모듈 디렉토리 제외
}
// 필터 생성 (숨김 파일 포함)
filter, err := files.NewFilter("", rules, true)
if err != nil {
return fmt.Errorf("필터 생성 실패: %v", err)
}

// 디렉토리 정보 가져오기
stat, err := os.Stat(dirPath)
if err != nil {
return fmt.Errorf("디렉토리 정보 가져오기 실패: %v", err)
}

// 필터를 사용하여 파일 생성
node, err := files.NewSerialFileWithFilter(dirPath, filter, stat)
if err != nil {
return fmt.Errorf("필터링된 파일 생성 실패: %v", err)
}
defer node.Close()

// 디렉토리인 경우 필터링된 내용 표시
if dir := files.ToDir(node); dir != nil {
```

```
fmt.Printf("필터링된 디렉토리 내용 (%s):\n", dirPath)
return listFilteredDirectory(dir, filter)
}

return nil
}

func listFilteredDirectory(dir files.Directory, filter *files.Filter) error {
entries := dir.Entries()
for entries.Next() {
entry := entries
node := entry.Node()
// 파일 정보 생성 (필터 테스트용)
fileInfo := &mockFileInfo{
name: entry.Name(),
mode: node.Mode(),
}
// 필터 적용 테스트
if filter.ShouldExclude(fileInfo) {
fmt.Printf("- %s [제외됨]\n", entry.Name())
continue
}
fmt.Printf("- %s [포함됨]", entry.Name())
```

```
if  files.ToDir(node) != nil {
fmt.Printf(" [디렉토리]")
}
fmt.Println()
}
return  entries.Err()
}
```

고급 활용

파일 시크(Seek) 기능

```
package main

import (
"fmt"
"io"

"github.com/ipfs/boxo/files"
)

func  demonstrateSeek() error {
// 큰 내용의 파일 생성
```

```
content := "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
file := files.NewBytesFile([]byte(content))
defer file.Close()

// 파일 중간으로 시크
offset := int64(10)
newPos, err := file.Seek(offset, io.SeekStart)
if err != nil {
return fmt.Errorf("시크 실패: %v", err)
}
fmt.Printf("시크 후 위치: %d\n", newPos)

// 현재 위치부터 10바이트 읽기
buffer := make([]byte, 10)
n, err := file.Read(buffer)
if err != nil && err != io.EOF {
return fmt.Errorf("읽기 실패: %v", err)
}
fmt.Printf("읽은 내용: %s\n", string(buffer[:n]))

// 파일 끝에서 5바이트 앞으로 시크
endPos, err := file.Seek(-5, io.SeekEnd)
if err != nil {
```

```
return fmt.Errorf("끝에서 시크 실패: %v", err)
}
fmt.Printf("끝에서 시크 후 위치: %d\n", endPos)

// 나머지 내용 읽기
remaining, err := io.ReadAll(file)
if err != nil {
return fmt.Errorf("나머지 읽기 실패: %v", err)
}
fmt.Printf("나머지 내용: %s\n", string(remaining))

return nil
}
```

멀티파트 파일 처리

```
package main

import (
"fmt"
"mime/multipart"
"strings"
```

```go
"github.com/ipfs/boxo/files"
)

func handleMultipartFile(multipartData string) error {
// 멀티파트 리더 생성
reader := multipart.NewReader(strings.NewReader(multipartData), "boundary")
// 멀티파트 파일 생성
multipartFile := files.NewMultipartFile(reader, nil)
defer multipartFile.Close()

// 멀티파트 파일이 디렉토리인지 확인
if dir := files.ToDir(multipartFile); dir != nil {
fmt.Println("멀티파트 파일 내용:")
entries := dir.Entries()
for entries.Next() {
entry := entries
fmt.Printf("- 파트: %s\n", entry.Name())
// 각 파트의 내용 읽기
if file := files.ToFile(entry.Node()); file != nil {
content, err := io.ReadAll(file)
if err != nil {
fmt.Printf(" 오류: %v\n", err)
continue
```

```
}
fmt.Printf(" 내용: %s\n", string(content))
}
}
return entries.Err()
}

return nil
}
```

모범 사례

1. 리소스 관리

```
func properResourceManagement(filePath string) error {
stat, err := os.Stat(filePath)
if err != nil {
return err
}

node, err := files.NewSerialFile(filePath, false, stat)
if err != nil {
return err
```

```
}
// 반드시 Close() 호출
defer func() {
if closeErr := node.Close(); closeErr != nil {
fmt.Printf("파일 닫기 오류: %v\n", closeErr)
}
}()

// 파일 처리 로직
return processNode(node)
}
```

2. 타입 안전성

```
func safeTypeHandling(node files.Node) error {
// 안전한 타입 변환
if file := files.ToFile(node); file != nil {
return handleFile(file)
} else if dir := files.ToDir(node); dir != nil {
return handleDirectory(dir)
} else {
return fmt.Errorf("지원되지 않는 파일 타입")
}
```

```
}

func handleFile(file files.File) error {
// 파일 처리 로직
fmt.Printf("파일 크기: %d\n", file.Size())
return nil
}

func handleDirectory(dir files.Directory) error {
// 디렉토리 처리 로직
fmt.Println("디렉토리 처리 중...")
return nil
}
```

3. 오류 처리

```
func robustFileProcessing(filePath string) error {
stat, err := os.Stat(filePath)
if err != nil {
if os.IsNotExist(err) {
return fmt.Errorf("파일이 존재하지 않습니다: %s", filePath)
}
return fmt.Errorf("파일 정보 가져오기 실패: %v", err)
```

```
}

node, err := files.NewSerialFile(filePath, false, stat)
if err != nil {
return fmt.Errorf("파일 생성 실패: %v", err)
}
defer node.Close()

// 파일 크기 확인
size, err := node.Size()
if err != nil {
return fmt.Errorf("파일 크기 확인 실패: %v", err)
}

// 큰 파일에 대한 처리
if size > 100*1024*1024 { // 100MB
fmt.Printf("경고: 큰 파일입니다 (%d 바이트)\n", size)
return processLargeFile(node)
}

return processRegularFile(node)
}
```

문제 해결

일반적인 오류와 해결책

1. **"file isn't a directory"**

- 디렉토리가 아닌 파일에 디렉토리 연산을 시도할 때 발생
- `files.ToDir()` 로 타입 확인 후 처리

2. **"file isn't a regular file"**

- 일반 파일이 아닌 객체에 파일 연산을 시도할 때 발생
- `files.ToFile()` 로 타입 확인 후 처리

3. **"operation not supported"**

- 지원되지 않는 연산을 시도할 때 발생
- 파일 타입별로 지원되는 연산 확인

성능 최적화

```
// 큰 파일 처리 시 버퍼링 사용
func optimizedFileReading(file files.File) error {
bufferSize := 64 * 1024  // 64KB 버퍼
buffer := make([]byte, bufferSize)
for {
n, err := file.Read(buffer)
if err == io.EOF {
break
}
```

```
if err != nil {
return err
}
// 버퍼 내용 처리
if err := processBuffer(buffer[:n]); err != nil {
return err
}
}
return nil
}
```

Files 모듈은 IPFS에서 다양한 소스의 파일을 통합적으로 처리할 수 있는 강력한 추상화를 제공합니다. 적절한 타입 확인과 리소스 관리를 통해 안전하고 효율적인 파일 처리가 가능합니다.